

TaskBoard Assessment

Anoop

May 19, 2026

TaskBoard Assessment — All Changes Made

Overview

This document records every change made to the TaskBoard codebase during the assessment, in the order they were completed. The work covers a code review, one critical bug fix, and two new features.

Part 1 — Code Review (REVIEW.md)

File created: REVIEW.md

A full code review was performed across all API routes, the database schema, and utility libraries. Three issues were found and documented, prioritized by business impact.

Issue 1 — IDOR: No Authorization on PATCH /api/tasks/[id] (Critical / Security)

File: src/app/api/tasks/[id]/route.ts · lines 16–38

What was wrong: The PATCH handler confirmed the user was authenticated but never checked whether the user was a member of the project that owned the task. Any logged-in user who knew a task ID could overwrite its title, status, assignee, or any other field. The DELETE handler in the same file correctly checked membership — PATCH was simply missed.

Proof of exploit:

```
# lina@example.com has no membership in Q3 Launch
curl -X PATCH "http://localhost:3001/api/tasks/TASK_ID" \
```

```
-H "Authorization: Bearer $TOKEN_LINA" \
-d '{"title":"HACKED","status":"done"}'
# Returns 200 – task is mutated
```

Recommended fix: Add `getProjectMembership + canEditTasks` check before the update.

Issue 2 — Missing `@unique` on `User.email` (*High / Data Integrity*)

File: `prisma/schema.prisma` · line 25

What was wrong: The email field had no database-level unique constraint. The registration endpoint guarded duplicates with an application-level `findFirst` check, but two concurrent requests could both pass before either write completed (TOCTOU race condition), creating two accounts with identical emails. Once duplicates exist, login becomes non-deterministic.

Recommended fix: Add `@unique` to the field and apply a migration. Switch `findFirst` → `findUnique` in auth routes.

Issue 3 — N+1 Over-fetch in `GET /api/projects` (*Medium / Performance*)

File: `src/app/api/projects/route.ts` · lines 10–21

What was wrong: The project listing loaded every Task row for every project a user was a member of, just to call `.length` on the result for `taskCount`. For a user in three projects with 1,000 tasks each, this transferred ~3,000 full task objects just to produce three numbers.

Recommended fix: Use Prisma's `_count` relation aggregation (`_count: { select: { tasks: true } }`), which emits a single `COUNT(*)` in the database.

Part 2 — Fix #1: IDOR on `PATCH /api/tasks/[id]`

File changed: `src/app/api/tasks/[id]/route.ts`

Added the missing membership and role check to the PATCH handler. The DELETE handler already had this logic — it was simply replicated for PATCH.

Lines added:

```
const membership = await getProjectMembership(user.id,
  existing.projectId);
if (!membership) return forbidden("you are not a member of this
  project");
if (!canEditTasks(membership.role))
  return forbidden("viewers cannot edit tasks");
```

Curl proof — before and after:

```
# BEFORE: viewer can mutate any task
curl -X PATCH "http://localhost:3001/api/tasks/TASK_ID" \
  -H "Authorization: Bearer $TOKEN_VIEWER"
# → 200 {"task": {"title": "HACKED", ...}}

# AFTER: viewer gets 403
curl -X PATCH "http://localhost:3001/api/tasks/TASK_ID" \
  -H "Authorization: Bearer $TOKEN_VIEWER"
# → 403 {"error": "viewers cannot edit tasks"}
```

Fix #3 — @unique Constraint on User.email**Files changed:**

- prisma/schema.prisma — added @unique
- prisma/migrations/20260519000002_unique_user_email/migration.sql — new migration
- src/app/api/auth/login/route.ts — findFirst → findUnique
- src/app/api/auth/register/route.ts — findFirst → findUnique

Schema change:

```
// Before
email String

// After
email String @unique
```

Migration SQL:

```
CREATE UNIQUE INDEX "users_email_key" ON "users"("email");
```

Fix #4 — N+1 Over-fetch in Project Listing

File changed: src/app/api/projects/route.ts

Before:

```
project: {
  include: {
    owner: { select: { id: true, name: true, email: true } },
    tasks: true, // ← loads ALL task rows for every project
  },
},
// ...
taskCount: m.project.tasks.length,
```

After:

```

project: {
  include: {
    owner: { select: { id: true, name: true, email: true } },
    _count: { select: { tasks: true } }, // ← single COUNT(*) per
      project
    },
  },
},
// ...
taskCount: m.project._count.tasks,

```

Part 3c — Airtable Bulk Export

New files:

File	Purpose
src/lib/airtable-client.ts	Real Airtable adapter — wraps the airtable npm package, matches the mock interface
src/lib/export-tasks.ts	Core export logic — retry, per-record error isolation, idempotency
src/app/api/projects/[id]/export/route.ts	POST API endpoint — admin/member only
src/tests/export-tasks.test.ts	8 unit tests using AirtableMockClient

UI change: src/app/projects/[id]/page.tsx — “export to airtable” button added to project header, visible to admin and member roles only.

How it works

1. POST /api/projects/:id/export is called from the UI button.
2. The endpoint authenticates the user, checks their role (viewer gets 403), then fetches all tasks for the project from the database (including assignee and createdBy relations).
3. The export logic in export-tasks.ts:
 - Calls client.list() to get all existing Airtable records and builds a Map<taskId, airtableRecordId>.
 - For each task: if already in the map → update(), else → create() with the task’s own ID stored in a TaskID field.
 - This makes every export **idempotent** — running it twice does not duplicate records.
4. Each operation is wrapped in withRetry():
 - Transient errors (429 rate limit, 500/502/503/504, network errors) → retry up to 3 times with exponential back-off (500 ms, 1 s, 2 s).
 - Permanent errors (422, 404, 400) → skip that record, record the error, continue with the rest.

- A single record failure never aborts the whole export.
5. Returns { exported, failed, errors }.

Task → Airtable field mapping

TaskBoard field	Airtable column	Type
id	TaskID	Single line text
title	Title	Single line text
description	Description	Long text
status	Status	Single line text
assignee.name	Assignee	Single line text
createdBy.name	CreatedBy	Single line text
position	Position	Number
projectId	ProjectId	Single line text
createdAt	CreatedAt	Single line text

Tests (src/tests/export-tasks.test.ts)

Test	What it checks
Creates a record per task	exported = 2 for two tasks
Maps all fields correctly	Each Airtable field matches the task's data
Idempotent on re-run	Second run updates, count stays at 1
Empty task list	Returns exported=0, failed=0
Skips failed record, continues	1 permanent failure → exported=2, failed=1
Retries transient errors	429 on first 2 attempts, succeeds on 3rd
Does not retry permanent errors	404 → called exactly once
Handles null description/assignee	Empty string fallback, no crash

Part 3a — Task Comments

New files:

File	Purpose
src/app/api/tasks/[id]/comments/route.ts	GET (list) and POST (create) endpoints
src/tests/comments.test.ts	15 unit tests
prisma/migrations/20260519000001_add_comments/migration.sql	Migration to create

File	Purpose
	comments table

Modified files:

File	What changed
prisma/schema.prisma	Added Comment model; added comments relation to Task and User
src/components/TaskDetail.tsx	Added comments thread + post form below the edit fields
src/app/projects/[id]/page.tsx	Passes currentUserRole to TaskDetail

Database model

```

model Comment {
  id          String   @id @default(cuid())
  taskId      String   @map("task_id")
  authorId    String   @map("author_id")
  body        String
  createdAt   DateTime @default(now()) @map("created_at")

  task Task @relation(fields: [taskId], references: [id], onDelete: Cascade)
  author User @relation(fields: [authorId], references: [id])

  @@index([taskId])
  @@map("comments")
}

```

API endpoints**GET /api/tasks/:id/comments**

- Auth required: yes
- Role required: any project member (admin, member, or viewer can read)
- Returns all comments for the task ordered chronologically (createdAt ASC)
- Each comment includes id, body, createdAt, and author (id, name, email)

POST /api/tasks/:id/comments

- Auth required: yes
- Role required: admin or member only (viewers get 403)
- Body: { "body": "string (1–10000 chars)" }
- Returns the created comment with author data
- Status: 201

No PATCH or DELETE routes exist — comments are append-only by design, enforcing the audit trail requirement.

Authorization matrix

Role	GET comments	POST comment
admin	✓	✓
member	✓	✓
viewer	✓	✗ (403)
non-member	✗ (403)	✗ (403)

Tests (src/tests/comments.test.ts)

GET tests (5):

Test	Expected
No auth token	401
Task does not exist	404
Caller not a project member	403
Valid member reads comments	200, ordered by createdAt ASC
Viewer can read comments	200

POST tests (8):

Test	Expected
No auth token	401
Task does not exist	404
Caller not a project member	403
Viewer tries to post	403 “viewers cannot post comments”
Admin posts comment	201
Member posts comment	201, correct authorId and body
Empty body	400
Missing body field	400

Append-only tests (2):

Test	Expected
No DELETE export	undefined
No PATCH/PUT export	undefined

Test Suite Summary

Test file	Tests	What it covers
auth.test.ts	2	JWT sign/verify round-trip

Test file	Tests	What it covers
schemas.test.ts	7	Zod validation for auth and task schemas
TaskCard.test.tsx	3	TaskCard component rendering
export-tasks.test.ts	8	Airtable export logic (Part 3c)
comments.test.ts	15	Comments API (Part 3a)
Total	35	All passing

Environment Setup

.env variables required:

```
DATABASE_URL="postgresql://taskboard:taskboard@127.0.0.1:5432/taskboard"
JWT_SECRET="dev-secret-change-me"
AIRTABLE_API_KEY="pat..."
AIRTABLE_BASE_ID="app..."
AIRTABLE_TABLE_NAME="Tasks"
```

App runs on: <http://localhost:3001> **Test credentials:** meera@taskboard.dev / password123